

List of Abbreviations

[illegible]

List of Symbols

Symbol	Full Name
N	FFT size / Number of points
n	Time-domain sample index
k	Frequency-domain subcarrier index
$x[n]$	Time-domain input sample
$X[k]$	Frequency-domain output sample
A	First butterfly input
B	Second butterfly input
W	FFT twiddle factor
Y_0	First butterfly output
Y_1	Second butterfly output
r	FFT radix
S	Number of FFT stages
j	Imaginary unit
$O(.)$	Algorithmic complexity notation
π	Mathematical constant pi
$\text{Re}(.)$	Real part
$\text{Im}(.)$	Imaginary part
L_{write}	Input transfer latency
L_{read}	Output retrieval latency
L_{compute}	Butterfly computation latency

Chapter 9

1 Hardware/ Software Co-Design

1.1 Introduction

In many signal-processing applications, a small subset of operations often dominates the overall execution time. Identifying these computational bottlenecks is a fundamental step in the hardware–software co-design process. Once identified, these critical operations can be redesigned as dedicated hardware accelerators and integrated into the system architecture to improve overall performance.

However, the effectiveness of such an approach depends not only on the computational efficiency of the accelerator itself, but also on the communication overhead introduced by the target platform and memory subsystem. As a result, accelerator design and system integration must be considered jointly to achieve meaningful performance gains. The methodology adopted for this hardware–software partitioning is presented in the chapter “**HW/SW Codesign**”, while the architectural features of the target platform are introduced in the chapters “**PULP Platform**” and “**Multi Cluster PULP**.”

The work presented in this thesis is motivated by the requirements of a MIMO-OFDM receiver, which serves as the target system investigated throughout this project. As discussed in the chapter “**MIMO in Mobile Communication Systems**,” modern wireless communication systems employ OFDM modulation to efficiently utilize the available spectrum and mitigate the effects of multipath propagation.

Within the receiver processing chain, the FFT is responsible for converting the received signals from the time domain to the frequency domain prior to channel equalization and symbol detection. In the considered system, this operation is performed repeatedly on large OFDM symbols and across multiple receive antennas, resulting in a substantial computational workload. Consequently, FFT processing becomes an attractive candidate for hardware acceleration.

To evaluate whether hardware acceleration can improve FFT execution on PULP, several butterfly accelerator architectures were designed and integrated into the PULP platform. Multiple butterfly architectures and radix implementations are developed and evaluated.

The integration of these accelerators into the PULP environment relies on the software infrastructure presented in “**Firmware**” and the architectural concepts discussed in “**PULP Platform**” and “**Multi Cluster PULP**.” Particular attention is given to the interaction between the processor, memory hierarchy, and accelerator interfaces, as these factors can significantly influence the overall system performance.

The resulting implementations are evaluated in terms of performance, communication overhead, and integration complexity. Therefore, the evaluation considers both performance improvement and the overhead introduced by accelerator integration into the PULP platform.

The insights obtained from this study provide a foundation for the hardware architectures presented later in the thesis, particularly in “**Hardware II: HWPE**,” “**Hardware III: System Pipeline**,” and “**System Performance**”

Analysis.” These chapters build upon the lessons learned from the FFT acceleration exploration to develop a more efficient system-level architecture for the target MIMO receiver.

1.1.1 Chapter Objectives

The objective of this chapter is to investigate the application of hardware/software co-design techniques to FFT acceleration on the PULP platform. The study focuses on identifying computational bottlenecks within FFT processing, developing dedicated hardware accelerators for butterfly operations, integrating these accelerators into the system architecture, and evaluating their impact on overall execution performance. Particular attention is given to the interaction between computational acceleration and communication overhead, as both factors contribute to the effectiveness of the final solution.

Objective	Description
FFT Analysis	Analyze the computational characteristics of FFT processing and identify performance bottlenecks.
Accelerator Design	Design dedicated hardware accelerators for FFT butterfly computations.
Platform Integration	Integrate the accelerators into the PULP architecture through different communication interfaces.
Software Support	Develop the software infrastructure required for accelerator control and data exchange.
Performance Evaluation	Evaluate the impact of architectural decisions on computational and system-level performance.

Table 1.1: Chapter objectives.

1.1.2 Chapter Contributions

The contributions of this chapter are summarized as follows:

- The development of several FFT butterfly accelerator architectures.
- The implementation and evaluation of different radix-based FFT approaches.
- The integration of the accelerators into the PULP platform through AXI and TCDM interfaces.
- The development of the software infrastructure required for accelerator control and data movement.
- A comprehensive performance analysis of the proposed hardware–software co-design solutions.

1.2 FFT Computational Analysis

1.2.1 FFT in the MIMO-OFDM Receiver

As described in the chapter “**MIMO in Mobile Communication Systems,**” the FFT is a fundamental component of the OFDM receiver chain. Following cyclic prefix removal, the received time-domain samples are transformed into the frequency domain before channel equalization and MIMO detection can be performed [ch9][1].

The transformation is given by the Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi kn/N} \quad (1.1)$$

where:

- $x[n]$: is the received time-domain sample,
- $X[k]$: is the frequency-domain output at subcarrier k ,
- N : is the FFT size,
- n : is the time-domain sample index,
- k : is the frequency-domain subcarrier index.

Direct computation of the DFT requires $O(N^2)$ operations. The FFT reduces this complexity to $O(N \log(N))$ through recursive decomposition into smaller operations known as butterflies [ch9][1].

1.3 Butterfly Computation as the Main Processing Element

The FFT achieves its computational efficiency by decomposing the DFT into a sequence of smaller operations known as butterflies. Rather than computing all frequency components directly, the FFT repeatedly combines intermediate results through butterfly computations distributed across multiple processing stages.

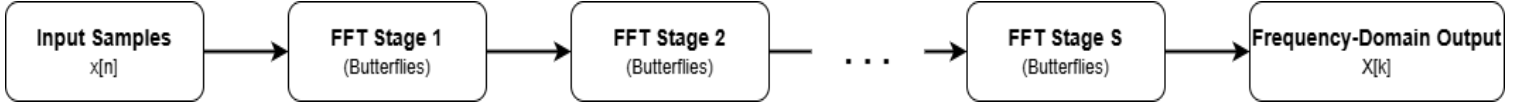


Figure 1.1: Butterfly Operations Distributed Across FFT Processing Stages

Regardless of the selected radix, the butterfly represents the fundamental computational element of the FFT. Each butterfly performs a combination of arithmetic operations, including additions, subtractions, and twiddle-factor multiplications. As these operations are executed repeatedly throughout the transform, butterfly processing accounts for a substantial portion of the overall FFT workload.

For an N -point FFT, the total number of butterfly computations grows with the transform size and the number of processing stages. Consequently, improvements in butterfly execution latency can have a direct impact on overall FFT performance.

This observation motivated the development of the butterfly accelerator architectures investigated in the following sections

1.4 Radix Selection and Design Space Exploration

The number of FFT stages depends on the selected radix according to [ch9] [1]

$$N_{stages} = \log_r(N) \quad (1.2)$$

where:

- N_{stages} : is the number of FFT stages,
- N : is the FFT size,
- r : is the FFT radix.

For the 4096-point FFT considered in this work, the stage count is shown in Table 1.1.

FFT Radix	Number of Stages
Radix-2	12
Radix-4	6
Radix-8	4

Table 1.2: FFT Stage Count for different radices

As shown in the previous, **increasing the FFT radix reduces the number of processing stages required to complete the transform**. Since each stage involves arithmetic operations, memory accesses, and synchronization events, reducing the stage count has the potential to improve overall execution efficiency.

However, the choice of radix also influences the structure and complexity of the corresponding butterfly operation. Consequently, the FFT radix represents an important architectural design parameter that affects both computational behavior and hardware implementation characteristics.

For this reason, multiple radix implementations were investigated in this work. The objective was to evaluate how different FFT decompositions influence performance when implemented as hardware accelerators on the PULP platform. The resulting butterfly architectures are presented later in “**Hardware I: Algorithms**” while their impact on system-level performance is evaluated in “**System Performance Analysis.**”

1.5 Computational and Communication Considerations

Although butterfly arithmetic constitutes a major portion of FFT execution, overall system performance is influenced by both **computation and data movement**. Hardware acceleration can reduce arithmetic latency, but the resulting performance gain also depends on the efficiency of communication between the processor, memory subsystem, and accelerator. Table 1.3 summarizes the primary contributors to FFT execution time [ch9][1].

Component	Impact on Performance
Butterfly Arithmetic	High
Twiddle Factor Access	Medium
Memory Transfers	High
Control Overhead	Medium

Table 1.3: Primary contributors to FFT execution

For this reason, both computation time and communication overhead are evaluated throughout this chapter, since accelerator performance and integration overhead jointly determine the achievable system-level improvement. The interaction between these factors ultimately determines the effectiveness of hardware acceleration within the PULP platform.

1.6 Computational Bottlenecks

The computational cost of FFT processing is largely dominated by complex multiplications performed within butterfly operations. A complex multiplication is defined as

$$(a + jb)(c + jd) = (ac - bd) + j(ad + cb) \quad (1.3)$$

where:

- (a, b, c) and (d) are real-valued operands,
- (j) is the imaginary unit.

This operation requires four real multiplications and two real additions, making it significantly more expensive than a simple addition or subtraction [ch9] [1].

Operation	Relative Cost
Addition/Subtraction	Low
Twiddle Factor Access	Medium
Complex Multiplication	High

Table 1.4: Major Contributors to FFT Computational Cost

The concentration of arithmetic complexity within the butterfly structure further motivates the use of dedicated hardware acceleration.

1.7 Hardware Accelerator Design

1.7.1 Design Objectives and Requirements

The FFT computational analysis presented in the previous section identified butterfly operations as the primary source of arithmetic complexity within the target OFDM receiver. Consequently, the objective of the hardware accelerator design was to reduce the execution time associated with butterfly computations while maintaining compatibility with the PULP platform.

Several design requirements were established prior to the accelerator development process:

- Compatibility with fixed-point arithmetic.
- Scalability to different FFT radix implementations.
- Low communication overhead between software and hardware.
- Sufficient throughput to justify hardware acceleration.

In addition to computational performance, accelerator efficiency was evaluated from a system-level perspective. As discussed in the “**Computational and Communication Considerations**” section, improvements in arithmetic latency do not necessarily translate directly into overall application speedup if communication and synchronization overheads dominate execution time.

Requirement	Description
Arithmetic Precision	Fixed-point complex arithmetic
FFT Support	Multiple radix structures
Platform Compatibility	PULP integration
Scalability	Support for larger FFT sizes
Performance	Reduced butterfly latency
Resource Efficiency	Moderate area utilization

Table 1.5: Hardware Accelerator Design Requirements

1.8 Radix-2 Butterfly Architecture

The radix-2 butterfly was selected as the baseline architecture due to its simplicity and widespread use in FFT implementations. A radix-2 FFT decomposes the transform into a sequence of two-point butterfly operations, requiring $\log_2(N)$ stages to complete an (N) -point FFT [ch9] [1].

The radix-2 butterfly performs one complex multiplication followed by two complex additions according to

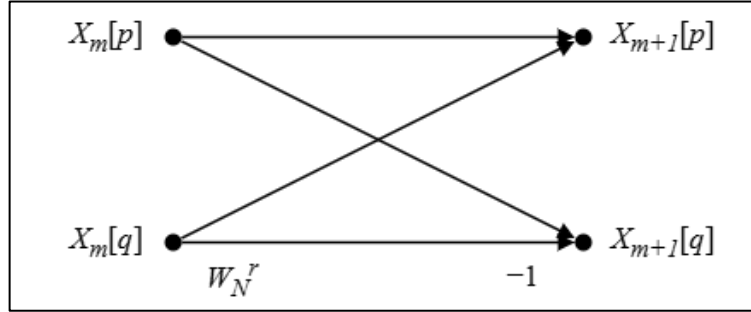


Figure 1.2: Radix-2 butterfly (adapted from [ch9][1])

$$Y_0 = A + BW \quad (1.4)$$

$$Y_1 = A - BW \quad (1.5)$$

where:

- A and B are complex input samples,
- W is the twiddle factor,
- Y0 and Y1 are the butterfly outputs.

The relatively simple arithmetic structure results in straightforward control logic and moderate hardware resource requirements. Furthermore, the butterfly can be easily pipelined, making it suitable for high-frequency implementations.

However, for a 4096-point FFT, a radix-2 decomposition requires twelve processing stages. **The large number of stages increases the amount of intermediate data movement and synchronization required during FFT execution**, which may limit the overall performance gains achievable through butterfly acceleration alone.

Parameter	Value
FFT Stages (4096 point)	12
Arithmetic Complexity	Low
Control Complexity	Low
Hardware Cost	Low
Data Movement	High

Table 1.6: Characteristics of the Radix-2 Butterfly

1.9 Radix-4 Butterfly Architecture

To reduce the number of FFT stages, a radix-4 butterfly architecture was also investigated. In a radix-4 decomposition, four input samples are processed together, reducing the number of required FFT stages **from twelve to $\log_4 N = 6$** [ch9][1].

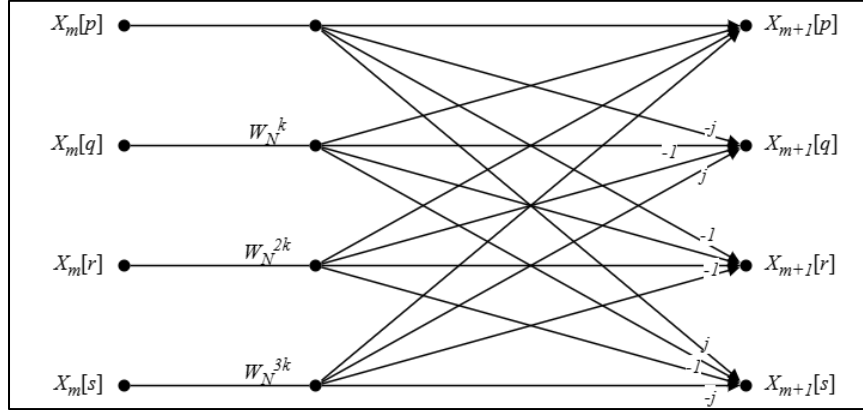


Figure 1.3: Radix-4 butterfly (adapted from [ch9][1])

By decreasing the number of stages, radix-4 FFTs reduce the amount of intermediate storage, memory traffic, and synchronization events required during execution. These characteristics make radix-4 an attractive compromise between computational efficiency and implementation complexity.

The reduction in stage count is achieved at the expense of a more complex butterfly structure. Compared to radix-2 implementations, radix-4 butterflies require additional arithmetic operations and more sophisticated control logic. Consequently, the hardware resource requirements increase, although the architecture remains practical for implementation within the target platform.

Parameter	Value
FFT Stages (4096-point)	6
Arithmetic Complexity	Medium
Control Complexity	Medium
Hardware Cost	Medium
Data movement	Lower

Table 1.7: Characteristics of the Radix-4 Butterfly

1.10 Radix-8 Butterfly Architecture

To further reduce the FFT stage count, a radix-8 butterfly architecture was evaluated. In this approach, eight input samples are processed within a single butterfly structure, reducing the required number of processing stages to four [ch9][1].

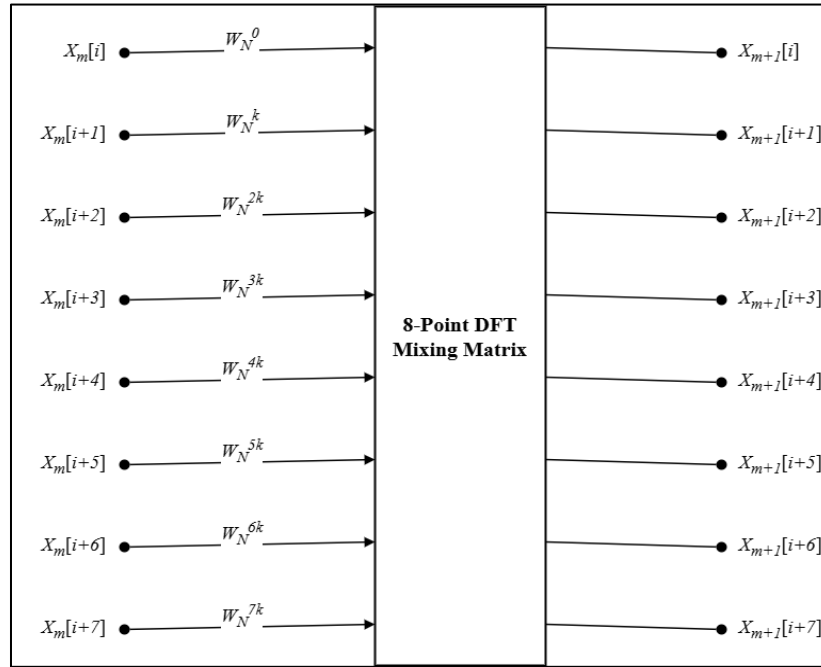


Figure 1.4: Radix-8 butterfly (adapted from [ch9][1])

The primary advantage of radix-8 FFTs is the significant reduction in stage count, which decreases synchronization overhead and limits the number of intermediate memory transactions. These properties can potentially improve system-level performance by reducing the amount of data movement required during FFT execution.

However, the radix-8 butterfly introduces substantially higher arithmetic complexity than radix-2 and radix-4 implementations. Additional arithmetic units, larger data paths, and more complex control structures are required to support the increased number of simultaneous computations. As a result, radix-8 implementations typically incur higher hardware costs and greater design complexity.

Parameter	Value
FFT Stages (4096-point)	4
Arithmetic Complexity	High
Control Complexity	High
Hardware Cost	High

Table 1.8: Characteristics of the Radix-8 Butterfly

1.11 Comparison of Investigated Radices

To evaluate the impact of FFT decomposition on accelerator performance, radix-2, radix-4, and radix-8 implementations were investigated. The three architectures provide different trade-offs between stage count, arithmetic complexity, and hardware resource utilization.

Metric	Radix-2	Radix-4	Radix-8
Stages (4096-point)	12	6	4
Arithmetic Complexity	Low	Medium	High
Control Complexity	Low	Medium	High
Hardware Cost	Low	Medium	High

Table 1.9: Comparison of Investigated FFT Radices

As the radix increases, the number of FFT stages decreases significantly. However, this reduction is accompanied by a corresponding increase in butterfly complexity and hardware resource requirements. The performance implications of these trade-offs are evaluated later in “**System Performance Analysis.**”

1.12 Hardware Integration into the PULP Architecture

1.12.1 Overview of the PULP SoC Domain

The FFT butterfly accelerators were initially developed as standalone hardware modules. Although these accelerators successfully performed the required arithmetic operations, they could not be directly controlled by software executing on the PULP platform. Consequently, an integration mechanism was required to connect the accelerators to the system infrastructure.

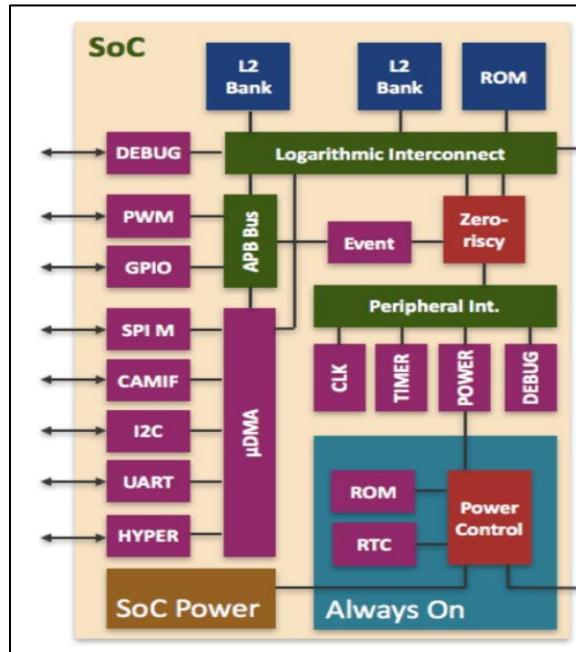


Figure 1.5: PULP SoC Domain ([ch9][2])

As described in “**PULP Platform**,” the PULP architecture combines programmable RISC-V cores with a shared memory hierarchy and a flexible communication network. Within the SoC domain, software interacts with memories and peripherals through memory-mapped transactions. This organization enables hardware components to be accessed using standard processor instructions [ch9] [2].

Communication between system components is performed through the AXI interconnect. Each peripheral is assigned a dedicated address range within the global memory map, allowing the interconnect to route transactions to the appropriate destination. This mechanism provides a standardized method for integrating custom hardware modules into the platform[ch9][2].

A key architectural characteristic of the PULP platform is the presence of two communication networks optimized for different workloads. As described in “**PULP Platform**” and “**Multi Cluster PULP**,” low-latency memory accesses are performed through the TCDM interconnect, while communication with peripherals and external hardware modules is handled through the AXI crossbar. The TCDM network is optimized for fast shared-memory transactions, whereas the AXI infrastructure provides a flexible, address-based mechanism for accessing system peripherals [ch9][2].

In the PULP communication model, hardware components are identified through dedicated memory addresses rather than explicit connections to software. Consequently, interaction with a hardware accelerator consists of reading from and writing to a predefined address range. This communication can be divided into a **control path**, used to configure and trigger accelerator execution, and a **data path**, used to transfer operands and retrieve computation results[ch9][2].

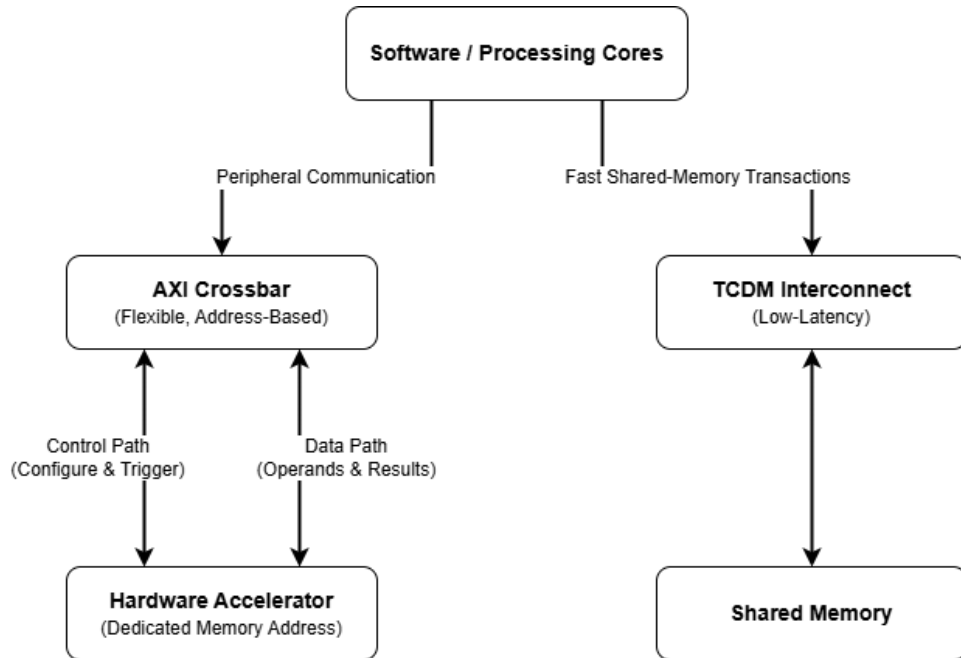


Figure 1.6: AXI and TCDM Communication Networks in the PULP Architecture

The objective of the integration process is therefore **to transform the FFT accelerator into a memory-mapped peripheral**. This step represents a key component of the hardware/software co-design methodology and represents a key component of the hardware/software co-design methodology adopted in this work.

1.13 AXI Slave Integration

1.13.1 Integration Philosophy

The FFT accelerators developed expose application-specific interfaces consisting of operand inputs, control signals, and computation outputs. While these interfaces are suitable for arithmetic processing, they are not compatible with the communication mechanisms used by software. An intermediate integration layer is therefore required[ch9][3].

To address this challenge, a wrapper-based integration approach was adopted. The accelerator is encapsulated inside a dedicated module that exposes a standard AXI slave interface toward the SoC while preserving the original data path internally. This approach avoids modifications to the computational core [ch9][3].

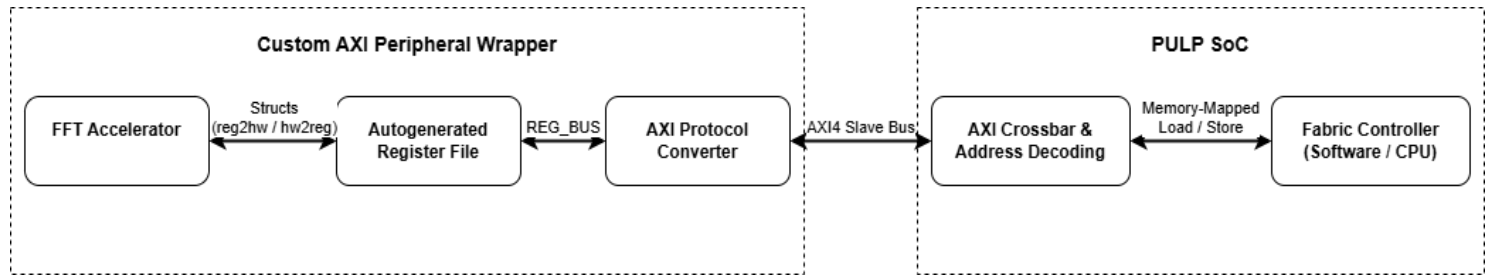


Figure 1.7: Architectural Layers of the FFT Accelerator Integration

The wrapper establishes a clear separation between communication and computation. Bus transactions are handled by the wrapper, while arithmetic processing remains the responsibility of the accelerator. This separation simplifies both development and verification [ch9][3].

1.13.2 Wrapper Architecture

The wrapper architecture is composed of three primary building blocks: the AXI interface, the register file, and the FFT accelerator. Together, these components establish a complete communication path between software and hardware. Each block performs a dedicated role within the integration process[ch9][3].

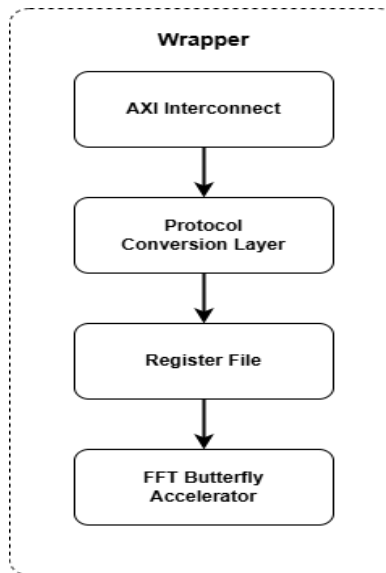


Figure 1.8: Internal organization of the accelerator wrapper. (Adapted from [ch9][3])

The AXI interface receives transactions originating from the processor and converts them into internal register accesses. These accesses may correspond to writing input operands, configuring execution parameters, or reading computation results. In this way, the complexity of the communication protocol is isolated from the accelerator.

The register file serves as the communication bridge between software and hardware. It stores configuration information, input operands, control signals, status flags, and output results. Through this register-based organization, software can interact with the accelerator using a familiar memory-mapped programming model.

The FFT accelerator remains focused exclusively on arithmetic computation. It operates on the information supplied through the register file and updates the corresponding output registers once execution is complete. This layered structure improves modularity and promotes design reuse.

Component	Function
AXI Interface	Receives and translates AXI transactions
Register File	Stores control, status, inputs, and outputs
FFT Accelerator	Performs butterfly computations

Table 1.10: Internal organization of the accelerator wrapper.

1.13.3 Register-Based Communication and Datapath Connectivity

Communication between software and the accelerator is performed through memory-mapped registers. Before execution begins, software writes the required operands and configuration parameters into dedicated input registers. A control register is subsequently used to initiate the accelerator operation[ch9][3].

Once execution starts, the accelerator processes the supplied data independently of the processor. During this period, software may monitor execution progress through status registers. These registers provide a simple synchronization mechanism between the processor and the hardware accelerator.

After completion, the generated outputs are stored in dedicated result registers. Software retrieves these values through standard memory read operations. From the processor perspective, interacting with the accelerator is therefore equivalent to accessing any other peripheral within the system [ch9][3].

Register Type	Purpose
Control	Execution control and configuration
Status	Completion and execution state
Input Registers	Butterfly operands
Output Registers	Computation results

Table 1.11: Accelerator Register Categories

The register file establishes the communication path between software and the FFT accelerator. Configuration parameters and input operands are transferred from the processor to the accelerator through dedicated registers. Similarly, results and status information are propagated from the accelerator back to software[ch9][3].

This organization allows the accelerator to remain entirely focused on arithmetic processing. Communication, synchronization, and protocol-related functionality are handled by the surrounding integration infrastructure. As a result, the computational core remains independent of system-level communication details.

An additional advantage of this approach is the separation of responsibilities between software and hardware. The processor manages data movement and execution control, while the accelerator performs the computational task. This division simplifies both debugging and verification activities.

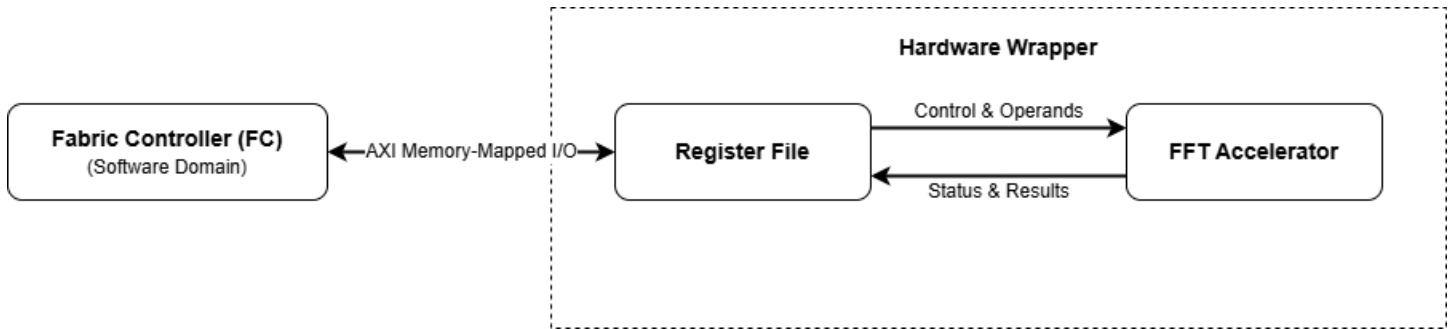


Figure 1.9: Data flow between software, register file, and FFT accelerator.

1.13.4 SoC-Level Integration

Once the wrapper architecture is completed, the accelerator can be connected to the SoC communication infrastructure. At this stage, the accelerator becomes a standard peripheral attached to the AXI interconnect. This allows software-generated transactions to reach the accelerator [ch9][3].

Integrating a new peripheral requires extending the interconnect with an additional AXI slave connection. A dedicated communication path is established between the accelerator wrapper and the AXI crossbar. This connection enables the accelerator to participate in the same communication framework used by the other peripherals in the system. However, physical connectivity alone is insufficient. The interconnect must also be

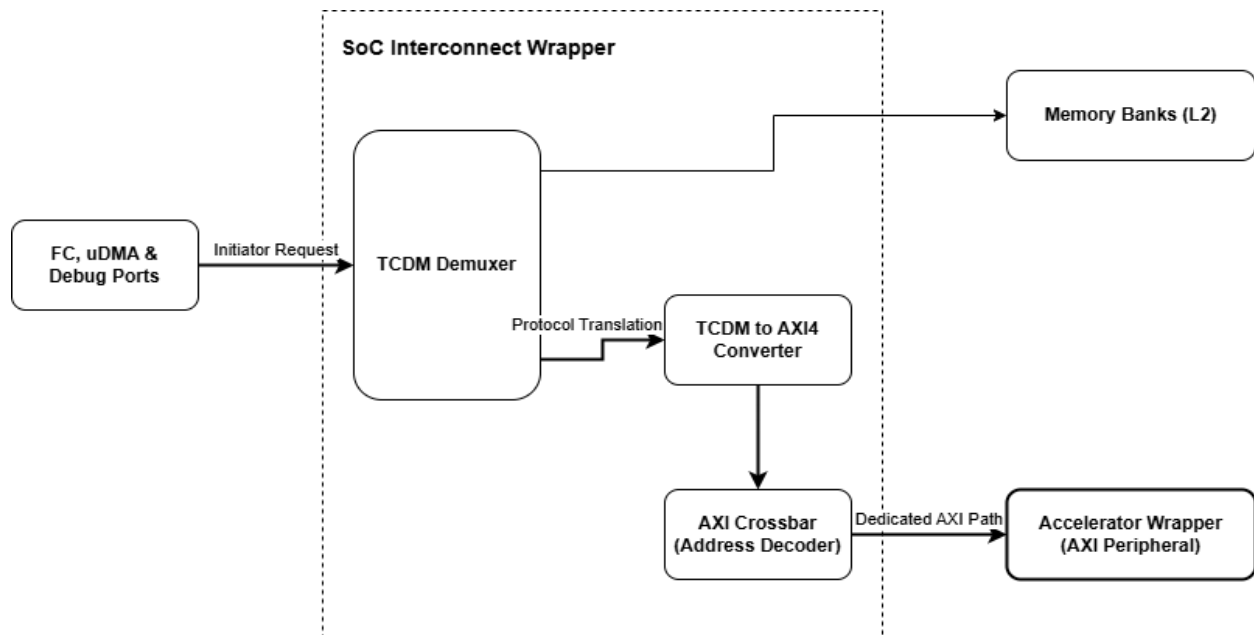


Figure 1.10: SoC Interconnect with the Integrated Accelerator([ch9][2])

capable of identifying which transactions are intended for the accelerator. This is provided through memory mapping and address decoding mechanisms[ch9][3].

1.13.5 Memory Mapping and Address Decoding

To make the accelerator accessible from software, a dedicated address range is assigned within the global SoC memory map. This address range uniquely identifies the peripheral and serves as the destination for all accelerator-related transactions. Care must be taken to ensure that the selected region does not overlap with existing memory or peripheral spaces [ch9][3].

The AXI interconnect maintains a set of address decoding rules that associate address regions with specific peripherals. Whenever the processor issues a memory transaction, the interconnect examines the target address and determines the appropriate destination. Transactions targeting the accelerator region are automatically routed to the accelerator wrapper [ch9][3].

From the software perspective, the FFT accelerator therefore appears as a conventional memory-mapped peripheral. All configuration, control, and result retrieval operations are performed through standard memory accesses. This preserves the programming model of the PULP platform while enabling hardware acceleration.

After applying the described integration steps, the FFT accelerator can be accessed directly from software through memory-mapped transactions. This provides the foundation for the software interaction mechanisms.

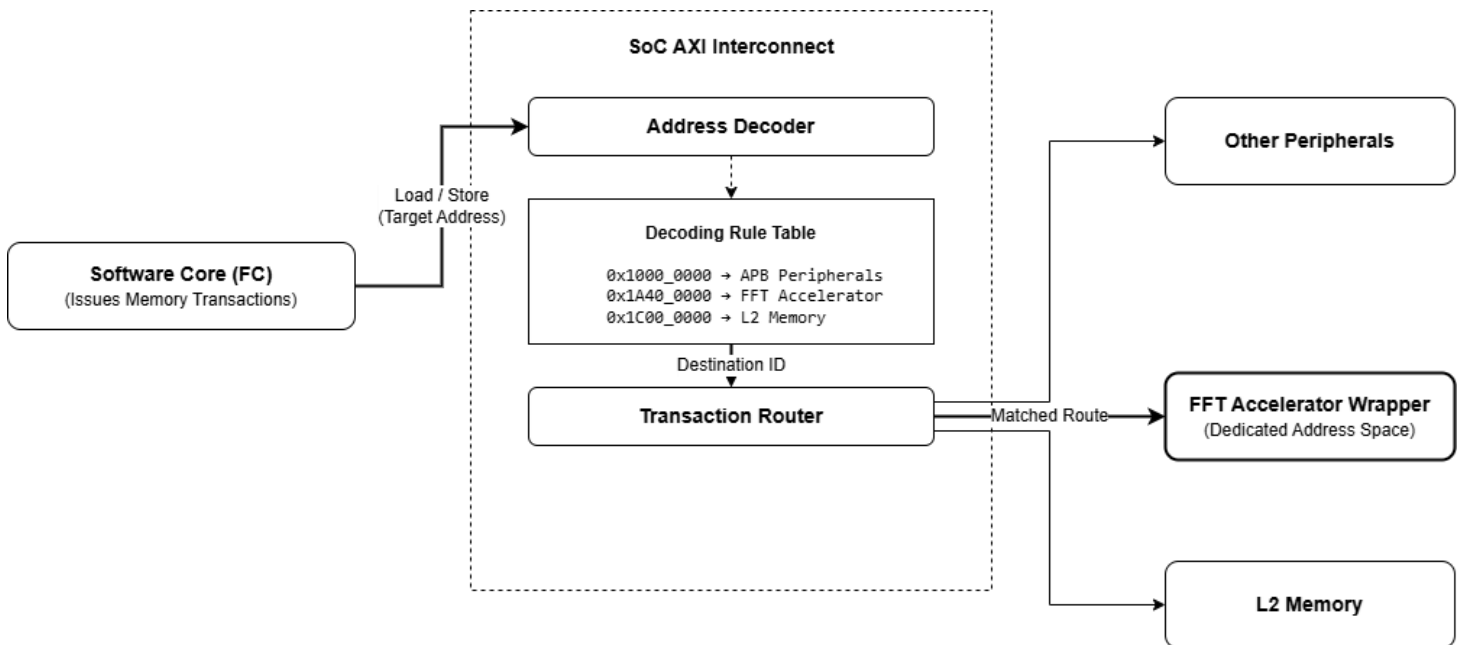


Figure 1.11: AXI address decoding and transaction routing within the SoC interconnect.

1.14 TCDM-Based Accelerator Integration

1.14.1 Motivation for TCDM Integration

Although the AXI-based integration approach provides a straightforward mechanism for connecting custom hardware accelerators to the PULP platform, communication overhead remains an important consideration. Each accelerator access requires transactions to traverse the AXI infrastructure and pass-through multiple protocol layers before reaching the computational core.

The effectiveness of hardware acceleration depends on whether the reduction in computation time outweighs the communication overhead introduced by integration. Consequently, reducing communication latency becomes an important objective in the hardware/software co-design process.

The PULP architecture provides an alternative communication mechanism through the TCDM interconnect. Unlike AXI-based peripherals, TCDM-connected components participate directly in the shared-memory infrastructure of the system. This organization enables lower-latency transactions and reduces the protocol overhead associated with peripheral communication.

1.15 TCDM Communication Model

The TCDM interconnect is designed to support low-latency communication between processing elements and shared memory resources. Rather than relying on a peripheral-oriented communication model, TCDM employs a lightweight transaction mechanism optimized for rapid data exchange.

From an architectural perspective, integrating an accelerator into the TCDM infrastructure transforms the accelerator into a shared-memory participant rather than an external peripheral. As a result, communication occurs through the same memory subsystem used by the processor cores.

This approach reduces the number of protocol conversion stages required for each transaction. Consequently, the communication path between software and hardware becomes shorter and more efficient.

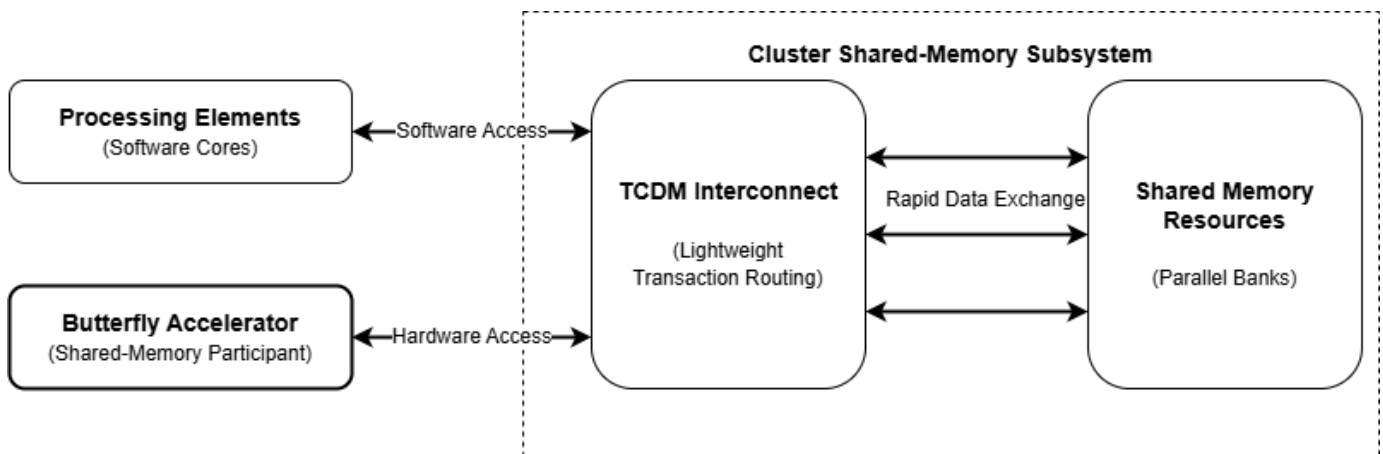


Figure 1.12: TCDM communication path for the butterfly accelerator.

1.16 Register-Based Interface Generation

Despite the change in communication infrastructure, software-accessible accelerator control remains based on memory-mapped registers. To maintain consistency between software and hardware representations of the accelerator interface, the register structure is described using a high-level register specification.

This register specification serves as a common description from which both hardware and software artifacts can be generated automatically. The generated hardware components implement register decoding and access control, while the generated software definitions provide symbolic register addresses and field descriptions.

A common register specification ensures consistency between the hardware implementation and the software driver. Furthermore, it reduces integration effort and minimizes the possibility of address mapping errors.

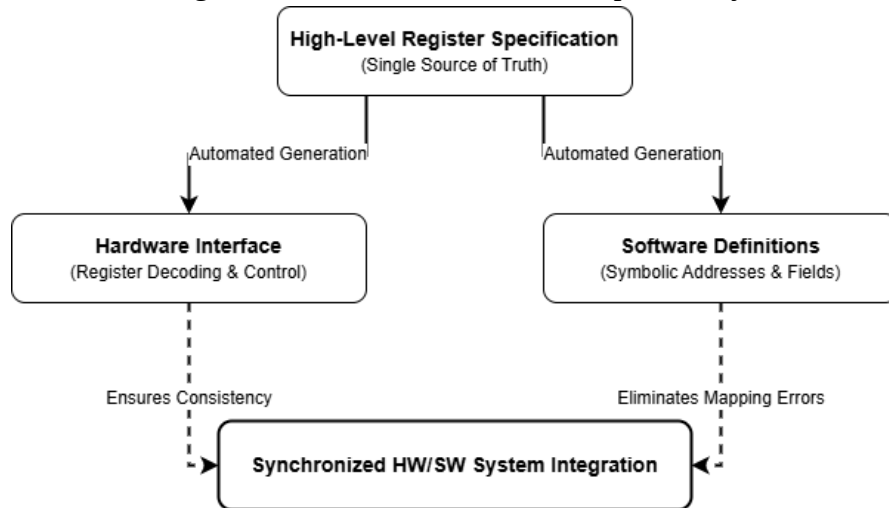


Figure 1.13: Register specification flow from description to generated hardware and software

1.17 Protocol Adaptation and Interface Translation

Although the accelerator is connected to the TCDM infrastructure, the register subsystem utilizes an internal register-access protocol. Consequently, a translation mechanism is required between the TCDM communication network and the register file implementation.

The protocol adaptation layer performs this function by converting TCDM transactions into the format expected by the register subsystem. Internally, the communication is represented using strongly typed request and response structures that simplify integration and improve interface consistency.

This layered approach separates communication concerns from computational functionality. The butterfly accelerator therefore remains focused exclusively on arithmetic processing, while protocol adaptation is handled by dedicated integration logic.

Layer	Function
TCDM Interface	Receives memory transactions
Protocol Translation Layer	Converts communication formats
Register File	Stores operands, control information, and results
Butterfly Accelerator	Performs FFT butterfly computations

Table 1.12: Functional layers of the TCDM-integrated accelerator.

1.18 SoC-Level Integration

Integrating the accelerator into the TCDM infrastructure requires modifications to the SoC interconnection network. These modifications establish a dedicated communication path between the processor subsystem and the accelerator.

At the architectural level, the integration process consists of three primary tasks. First, the accelerator must be instantiated within the SoC hierarchy. Second, the communication infrastructure must be extended to include a new TCDM endpoint. Third, routing logic must be updated so that transactions targeting the accelerator address space are forwarded to the newly integrated component.

These modifications allow the accelerator to participate directly in the shared-memory communication network while remaining accessible through the standard system addressing model.

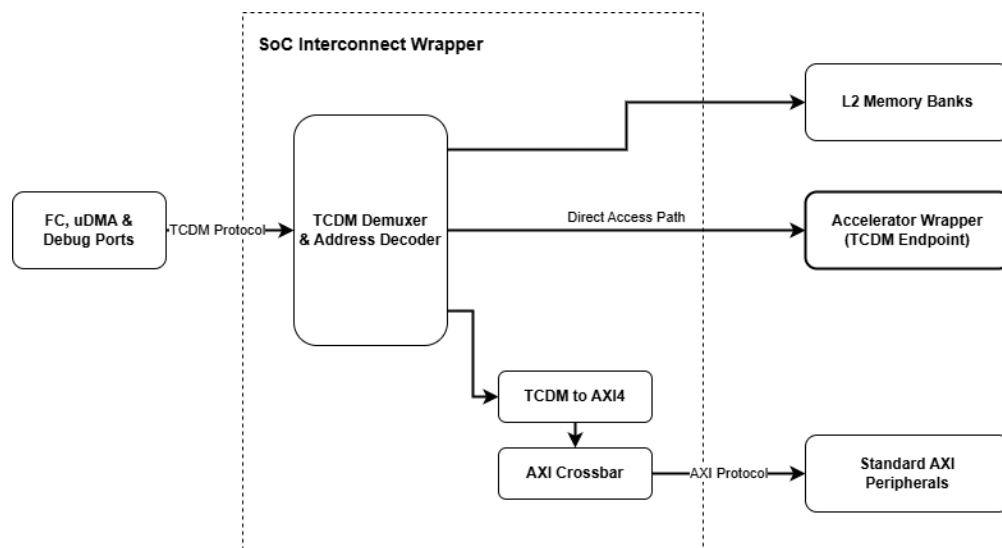


Figure 1.14: Integration of the butterfly accelerator into the TCDM interconnect.

1.19 Address Mapping and Transaction Routing

To enable software access, a dedicated address region is assigned to the accelerator within the system memory map. This address space uniquely identifies the accelerator and prevents conflicts with existing memories and peripherals.

When a processor issues a memory transaction, the interconnect examines the target address and determines the appropriate destination. Transactions whose addresses fall within the accelerator region are routed toward the TCDM-connected butterfly module. Transactions targeting other address ranges continue to follow their normal routing paths. This mechanism preserves the standard memory-mapped programming model while enabling direct communication through the TCDM infrastructure.

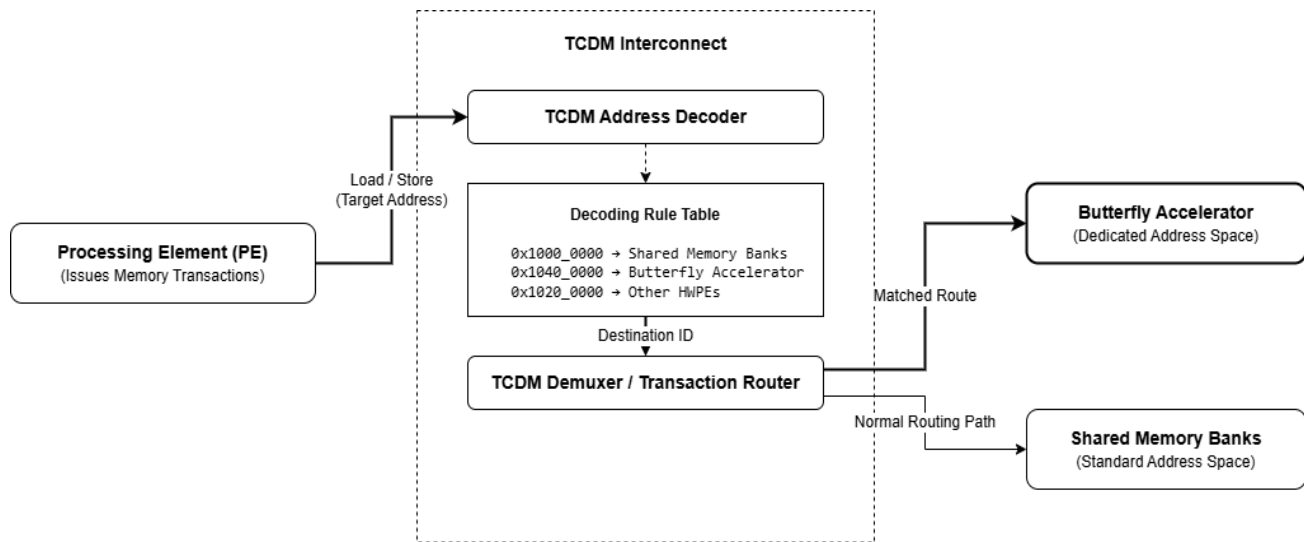


Figure 1.15: Address decoding and transaction routing for the TCDM-integrated accelerator.

1.20 Comparison of AXI and TCDM Integration

Both AXI and TCDM integration enable the butterfly accelerator to be accessed through a memory-mapped interface. However, the two approaches differ in how transactions are transported between the processor and the accelerator.

The AXI-based implementation treats the accelerator as a conventional peripheral connected through the SoC communication infrastructure. This approach provides a standardized and modular integration methodology but introduces additional protocol processing during communication.

In contrast, the TCDM-based implementation connects the accelerator directly to the shared-memory interconnect. As a result, communication occurs through a shorter transaction path with reduced protocol overhead.

Table 1.13 [ch9] summarizes the main differences between the two integration approaches.

Feature	AXI Integration	TCDM Integration
Integration Type	Peripheral-based	Shared-memory based
Communication Path	AXI Crossbar	TCDM Interconnect
Protocol Complexity	Higher	Lower
Communication Latency	Higher	Lower
Integration Effort	Lower	Higher
Transaction Overhead	Significant	Reduced

Table 1.13: Comparison between AXI and TCDM integration.

From a hardware/software co-design perspective, the comparison highlights the importance of considering communication overhead in addition to computational acceleration. While both approaches provide access to the same computational hardware, the surrounding communication infrastructure can significantly influence overall system performance. The quantitative impact of these differences is evaluated later in this chapter.

1.21 Software Support Infrastructure

Regardless of the selected integration approach, software interacts with the FFT accelerator through a memory-mapped register interface. Although AXI and TCDM differ in their underlying communication mechanisms, both support the same software interaction model. A software abstraction layer is therefore used to provide configuration, execution control, status monitoring, and result retrieval.

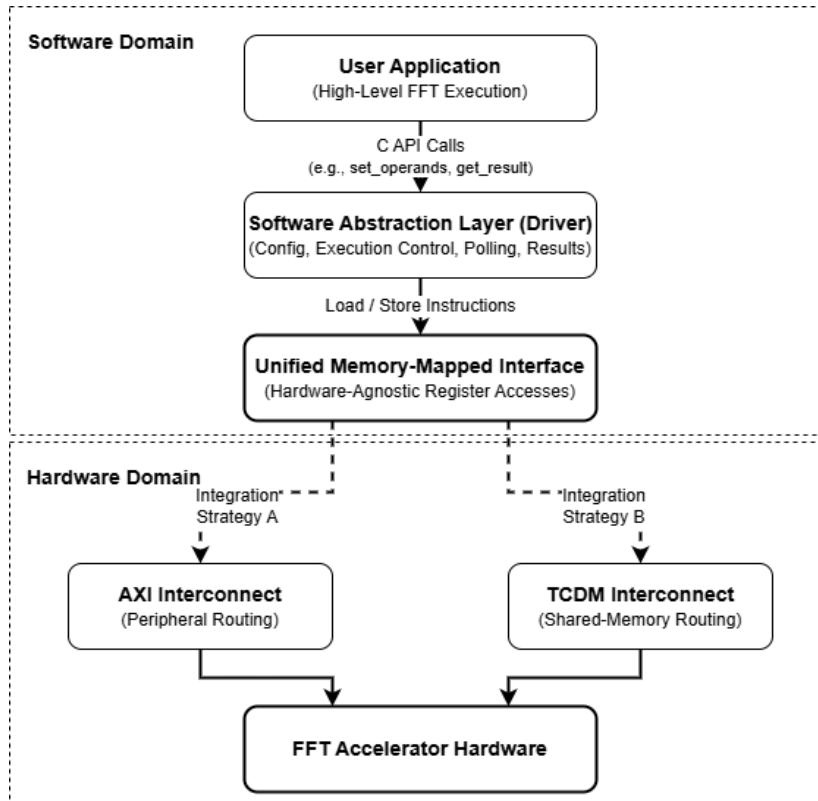


Figure 1.16: Software abstraction layer for AXI- and TCDM-based accelerator integration.

1.22 Driver-Based Accelerator Access

Direct access to hardware registers from application code reduces portability and exposes implementation-specific details. To address this issue, a dedicated software driver is used as an abstraction layer between software and hardware.

The driver encapsulates the communication mechanisms required to interact with the accelerator while providing a uniform programming interface. As a result, application software can invoke accelerator functionality without requiring knowledge of the underlying communication infrastructure.

Driver Functionality	Purpose
Configuration Management	Configure accelerator parameters
Operand Transfer	Transfer input data to the accelerator
Execution Control	Trigger hardware execution
Status Monitoring	Detect completion of computation
Result Retrieval	Read generated results

Table 1.14: Functional responsibilities of the accelerator driver.

1.23 Register Abstraction

The software interface relies on a register abstraction mechanism that provides symbolic access to hardware resources. Rather than using manually defined addresses, software accesses accelerator registers through automatically generated register definitions derived from a common register specification. This approach ensures consistency between the software-visible interface and the hardware implementation. Furthermore, it reduces integration effort and minimizes the possibility of address mapping errors.

1.24 Runtime Execution Model

During operation, control-intensive tasks remain under software management while computationally intensive butterfly operations are delegated to dedicated hardware. This partitioning follows the fundamental principles of hardware/software co-design by assigning each component to the domain most suitable for its execution characteristics.

The execution process begins with accelerator configuration and operand transfer. Once the required data has been provided, software initiates execution through the control interface. The accelerator then performs the butterfly computation independently before returning the generated results through the same memory-mapped interface.

This execution model remains identical for both AXI- and TCDM-based implementations, ensuring a consistent software environment while allowing different hardware communication architectures to be evaluated.

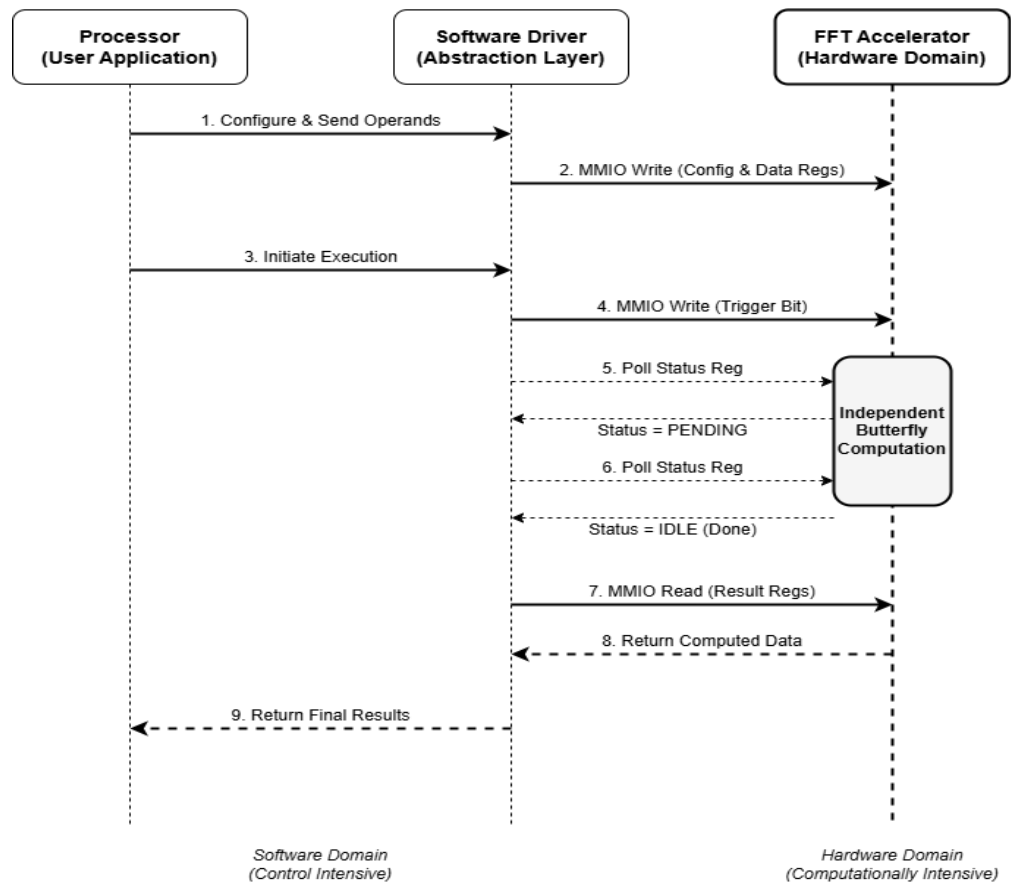


Figure 1.17: Runtime interaction between the processor, software driver, and butterfly

1.25 Experimental Evaluation of AXI-Based Integration

Following the integration methodology presented in "**AXI Slave Integration**", a series of experiments was conducted to evaluate the impact of communication overhead on the performance of the butterfly accelerator. The objective of these experiments was to progressively optimize the AXI-based implementation and identify the factors that most significantly influence end-to-end execution latency.

All measurements reported in this section correspond to complete accelerator execution, including software interaction, data transfers, control operations, and hardware computation.

1.25.1 Experiment 1: Baseline AXI Implementation

1.25.1.1 Experimental Setup

The first experiment established the baseline AXI implementation used for subsequent optimization studies. The butterfly accelerator was integrated as a memory-mapped peripheral through the AXI interconnect using the wrapper architecture described previously in "**Wrapper Architecture**".

In this implementation, software transferred the complete twiddle-factor value together with the butterfly operands through the AXI interface. Furthermore, accelerator execution relied on explicit software control, requiring dedicated transactions for configuration and execution management.

The arithmetic Datapath employed the corrected operand widths adopted for the final butterfly architecture:

- 24-bit inputs, 24-bit outputs
- 32-bit twiddle factors.

1.25.1.2 Results

The measured end-to-end latency was

$$L_{axi\ total} = 59\ cycles \quad (1.6)$$

where:

- $L_{axi\ total}$ is the measured end-to-end execution latency including measurement overhead.

1.25.1.3 Discussion

Although the accelerator successfully performed the required butterfly computation, the measured latency was significantly larger than the computational latency of the arithmetic core itself.

This observation suggested that communication and software interaction represented a substantial portion of the overall execution time. Consequently, further optimization efforts focused on reducing communication overhead rather than modifying the butterfly arithmetic.

1.25.2 Experiment 2: Communication-Aware AXI Optimization

1.25.2.1 Objective

The objective of the second experiment was to reduce the amount of information exchanged between software and hardware while preserving the same computational functionality.

1.25.2.2 Twiddle-Factor Transfer Optimization

In the baseline implementation, software transferred the complete twiddle coefficient for every butterfly operation. Since the required twiddle values were already available within the hardware design, repeatedly transmitting the full coefficient introduced unnecessary communication overhead.

To address this limitation, the communication mechanism was modified so that software transmitted only the twiddle-factor index. The corresponding coefficient was then selected internally by the accelerator.

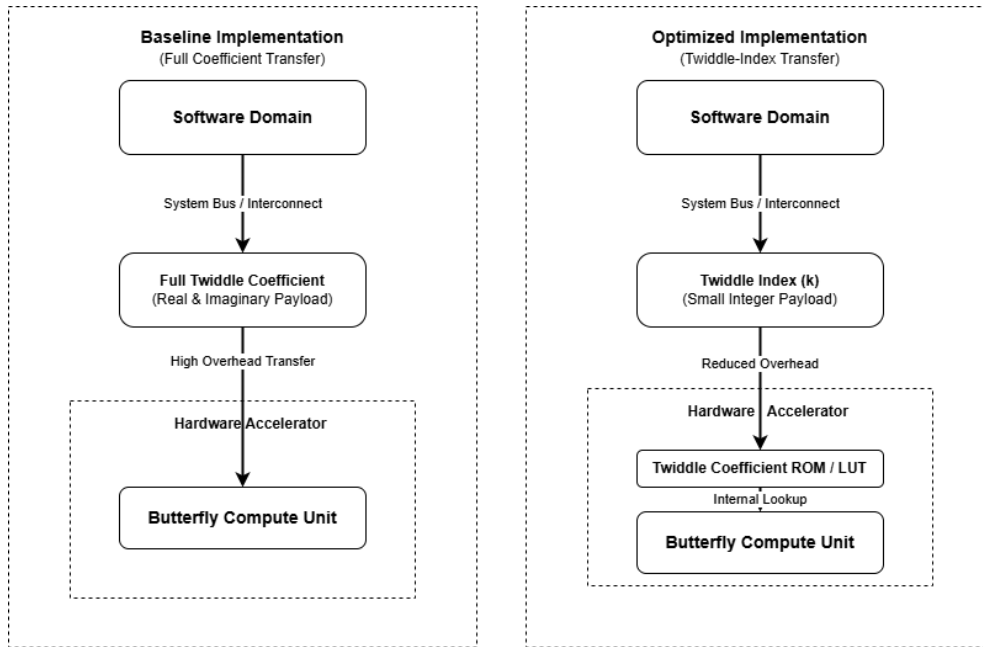


Figure 1.18: Comparison between full twiddle-factor transfer and twiddle-index transfer.

1.25.2.3 Wrapper and Driver Optimization

Additional optimizations were introduced in both the accelerator wrapper and the software driver.

The wrapper was simplified to reduce transaction overhead, while the software driver was modified to improve the efficiency of register accesses and accelerator invocation. These modifications reduced the communication cost without altering the butterfly computation itself.

1.25.2.4 Results

The measured end-to-end latency was reduced to

$$L_{axi\ total} = 24\ cycles \quad (1.7)$$

1.25.2.5 Discussion

Compared to the baseline implementation, the latency decreased from 59 cycles to 24 cycles.

Since the butterfly architecture remained unchanged, the observed improvement originated primarily from communication-related optimizations. These results demonstrate that reducing transferred data volume can significantly improve accelerator performance even when the computational hardware remains identical.

1.25.3 Experiment 3: Fully Optimized AXI Implementation

1.25.3.1 Objective

The objective of the third experiment was to minimize the number of AXI transactions required for each butterfly execution.

While Experiment 2 reduced the amount of transferred information, several control-related transactions were still required to initiate execution and manage accelerator status.

1.25.3.2 Auto-Trigger Optimization

In the previous implementation, software executed three separate operations:

1. Write the first operand.
2. Write the second operand.
3. Write a control value to initiate execution.

Although the control transaction carried very little information, it still required a complete AXI transaction.

To eliminate this overhead, an automatic triggering mechanism was introduced. Instead of waiting for an explicit start command, the accelerator automatically began execution once the final operand had been written.

This modification **removed one AXI transaction from every butterfly operation.**

1.25.3.3 Auto-Status Optimization

The baseline implementation also required software to manage accelerator status explicitly. After each operation, software performed additional register accesses to clear and update status information before the next computation could begin. To reduce this overhead, automatic status management was introduced. Status information was updated directly by the hardware interface, eliminating the need for dedicated software transactions.

As with the auto-trigger optimization, the butterfly arithmetic remained unchanged.

1.25.3.4 Architectural Refinement

The final implementation also employed a flattened architecture that reduced unnecessary wrapper overhead while preserving the original functionality of the accelerator. These modifications produced the most efficient AXI implementation investigated in this work.

1.25.3.5 Results

The measured end-to-end latency was

$$L_{axi\ total} = 24\ cycles \quad (1.8)$$

This represents the lowest latency achieved using AXI integration.

1.25.4 Detailed Latency Analysis

To better understand the remaining performance limitations, the fully optimized AXI implementation was analyzed at the transaction level. **Figure 1.19**[ch9] presents the timing diagram of the complete accelerator execution, including operand transfers, butterfly computation, and result retrieval.

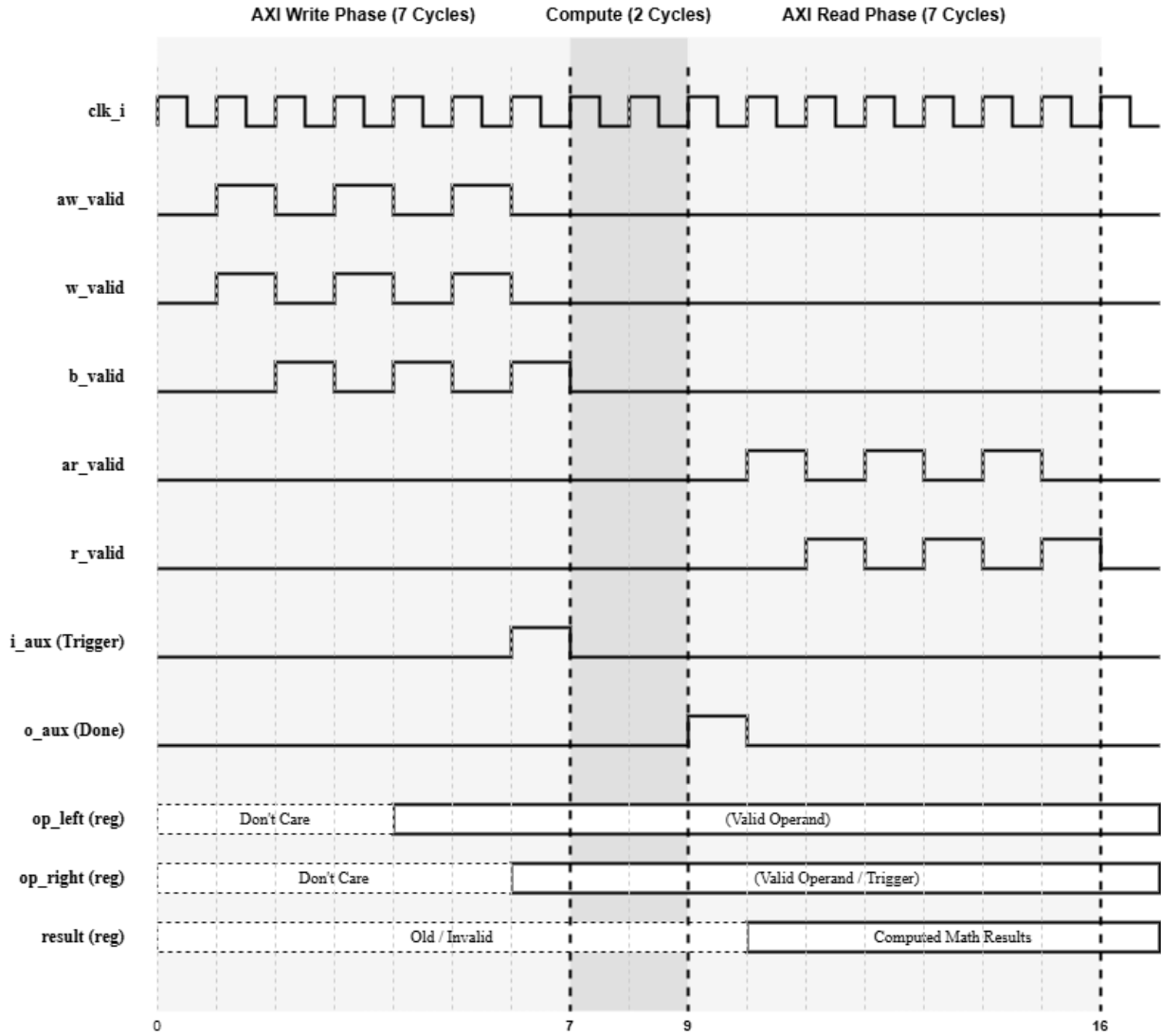


Figure 1.19: Simulation results of the AXI-integrated butterfly accelerator

As shown in **Figure 1.19** [ch9], the execution process can be divided into three distinct phases. The first phase corresponds to writing the twiddle index and input operands through the AXI interface. Once all required inputs have been transferred, the butterfly accelerator begins computation automatically through the auto-trigger mechanism. After the computation completes, the generated outputs are retrieved through a sequence of AXI read transactions.

Based on this execution flow, the total latency can be expressed as

$$L_{axi} = L_{write} + L_{compute} + L_{read} \quad (1.9)$$

where:

- L_{axi} is the accelerator execution latency,
- L_{write} is the input transfer latency,
- $L_{compute}$ is the butterfly computation latency,
- L_{read} is the output retrieval latency.

The timing diagram shows that the write phase requires seven clock cycles. These cycles correspond to the transfer of the twiddle index and the two complex input operands through the AXI interface. The butterfly computation occupies only two clock cycles. During the first cycle, intermediate values are generated, while the second cycle produces the final butterfly outputs. Following computation, the read phase requires seven additional clock cycles to retrieve the generated results and complete the software-hardware interaction.

Operation	Latency (Cycles)
AXI Write Phase	7
Butterfly Computation	2
AXI Read Phase	7
Total Accelerator Latency	16

Table 1.15: Latency breakdown of the fully optimized AXI implementation.

Therefore,

$$L_{axi} = 7 + 2 + 7 = 16 \text{ cycles} \quad (1.10)$$

Performance measurements were obtained using hardware performance counters. The measurement procedure requires one cycle to start the counter and one cycle to stop it. Consequently, the measured end-to-end latency becomes

$$L_{axi \text{ total}} = 16 + 2 = 18 \text{ cycles} \quad (1.11)$$

where:

- $L_{axi \text{ total}}$ is the measured end-to-end execution latency including measurement overhead.

A notable observation is that the butterfly computation itself occupies only two cycles, whereas fourteen cycles are consumed by communication and data movement. This result indicates that communication overhead remains the dominant contributor to total execution latency despite the extensive AXI optimizations introduced in Experiment 3.

1.25.4.1 Discussion

A particularly important observation is that the butterfly computation itself requires only two clock cycles, while the complete accelerator invocation requires eighteen clock cycles.

Consequently, the majority of the execution time is associated with communication and software interaction rather than arithmetic processing. This result highlights the importance of considering communication overhead alongside computational performance when evaluating hardware accelerators within a hardware/software co-design framework.

1.26 Experimental Evaluation of TCDM-Based Integration

Following the integration methodology presented in "**TCDM-Based Accelerator Integration**", a second set of experiments was conducted to evaluate the impact of connecting the butterfly accelerator directly to the TCDM interconnect. Unlike the AXI-based implementation, the TCDM approach allows the accelerator to communicate through the shared-memory infrastructure of the PULP platform, thereby reducing the protocol overhead associated with peripheral-based communication.

The objective of these experiments was to determine how the reduced communication overhead provided by the TCDM interconnect influences the overall execution latency of the butterfly accelerator while maintaining the same computational functionality.

1.26.1 Experiment 1: Initial TCDM Integration

1.26.1.1 Experimental Setup

The first TCDM experiment migrated the butterfly accelerator from the AXI communication to the TCDM interconnect. The computational core remained unchanged, ensuring that any performance differences originated from the communication architecture rather than modifications to the butterfly implementation.

As shown in the experimental configuration summary, the implementation employed the same corrected operand widths, twiddle-index delivery mechanism, and standard multiplier used in the final AXI implementation. However, no wrapper optimizations were applied and the software driver retained a high-overhead execution model.

The software interaction flow also remained unchanged. Input operands were written through memory-mapped registers, the accelerator was triggered through software control, and the generated results were retrieved through the same register interface.

1.26.1.2 Results

The measured end-to-end latency was

$$L_{tcdm\ total} = 48\ cycles \quad (1.12)$$

where:

- L_{tcdm} is the total execution latency of the TCDM-integrated butterfly accelerator.

1.26.1.3 Discussion

Although the accelerator was now connected directly to the TCDM infrastructure, the measured latency remained significantly larger than the computational latency of the butterfly core itself.

This observation indicates that communication overhead is not determined solely by the underlying interconnect. Software interaction and wrapper design also contribute significantly to the overall execution time. Consequently, additional optimization efforts focused on reducing these remaining overheads.

1.26.2 Experiment 2: Optimized TCDM Implementation

1.26.2.1 Objective

The objective of the second experiment was to reduce the remaining software and interface overheads while preserving the same butterfly functionality.

1.26.2.2 Wrapper Optimization

The initial TCDM implementation employed a straightforward integration strategy intended primarily to verify functionality and correctness. Although mathematically correct, the communication path still contained unnecessary overhead associated with software-hardware interaction.

To improve efficiency, the wrapper architecture was optimized to simplify communication between the processor and the butterfly accelerator. These modifications reduced transaction overhead without altering the computational behavior of the butterfly core.

1.26.2.3 Software Optimization

Additional optimizations were introduced within the software driver. The communication sequence was streamlined and unnecessary software overhead was eliminated.

Since both AXI and TCDM implementations expose the accelerator through the same memory-mapped software interface, several of the software optimizations previously introduced for the AXI implementation could be reused directly in the TCDM environment.

1.26.2.4 Results

The measured end-to-end latency was reduced to

$$L_{tcdm\ total} = 11\ cycles \quad (1.13)$$

1.26.2.5 Discussion

Compared to the initial TCDM implementation, the latency decreased from 48 cycles to 11 cycles.

As in the AXI experiments, the butterfly arithmetic remained unchanged throughout both implementations. Therefore, the observed improvement can be attributed primarily to reductions in communication and software overhead rather than modifications to the computational core.

1.27 Detailed Latency Analysis

To better understand the remaining performance limitations, the optimized TCDM implementation was analyzed at the transaction level. **Figure 1.20** presents the timing diagram of the complete accelerator execution.

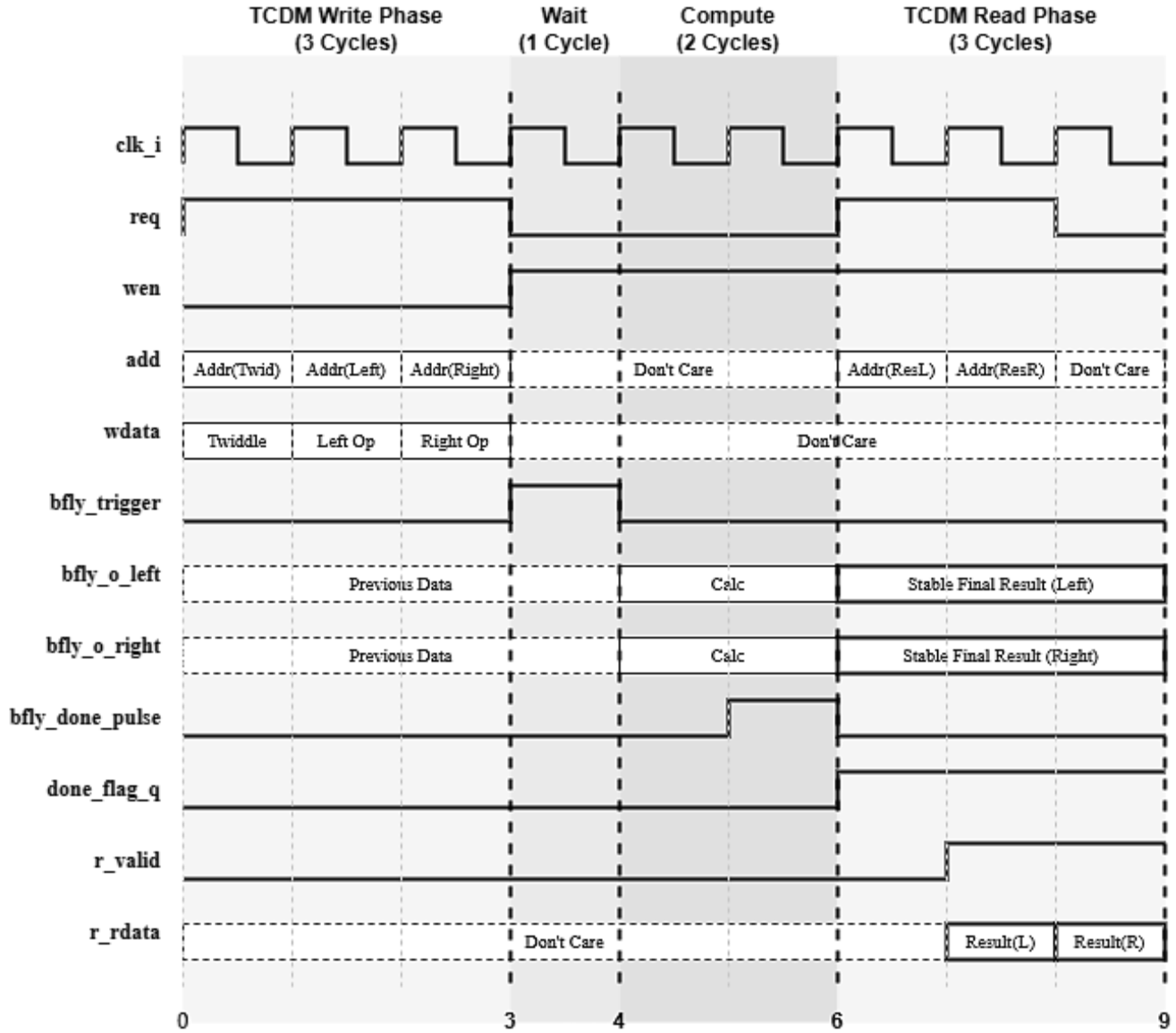


Figure 1.20: Simulation results of the TCDM-integrated butterfly accelerator.

As shown in **Figure 1.20**[ch9], the execution process can be divided into three distinct phases. The first phase corresponds to writing the twiddle index and input operands through the TCDM interface. Once the required inputs become available, the butterfly computation is performed. Finally, the generated outputs are retrieved through TCDM read transactions.

Based on this execution flow, the total latency can be expressed as

$$L_{tcdm} = L_{write} + L_{compute} + L_{read} \quad (1.14)$$

where:

- L_{tcdm} is the accelerator execution latency,

- L_{write} is the input transfer latency,
- $L_{compute}$ is the butterfly computation and synchronization latency,
- L_{read} is the output retrieval latency.

The timing analysis shows that the write phase requires three clock cycles. These cycles correspond to writing the twiddle index and the two complex operands through the TCDM interface. Unlike AXI communication, each write operation can be completed in a single cycle.

The computation phase occupies three clock cycles. Two cycles are required for the butterfly arithmetic, while an additional synchronization cycle is introduced to align the generated outputs with the TCDM communication interface.

The read phase requires three additional clock cycles to retrieve the generated outputs.

Operation	Latency (Cycles)
TCDM Write Phase	3
Butterfly Computation and Synchronization	3
TCDM Read Phase	3
Total Accelerator Latency	9

Table 1.16: Latency breakdown of the optimized TCDM implementation.

Therefore,

$$L_{tcdm} = 3 + 3 + 3 = 9 \text{ cycles} \quad (1.15)$$

Performance measurements were obtained using hardware performance counters. Similar to the AXI implementation, the measurement start and stop operations introduce an additional two clock cycles.

The final measured latency therefore becomes

$$L_{tcdm \text{ total}} = 9 + 2 = \text{cycles} \quad (1.16)$$

where:

- $L_{tcdm \text{ total}}$ is the measured end-to-end execution latency including measurement overhead.

1.27.1 Discussion

The latency analysis shows that the optimized TCDM implementation brings the measured execution time significantly closer to the intrinsic latency of the butterfly core. While communication and synchronization overhead remain present, their contribution is substantially smaller than that observed in the AXI-based implementation. Consequently, a larger fraction of the total execution time is devoted to useful computation.

1.27.2 Experiment 3: Optimized TCDM Implementation with Radix-8 Butterfly

1.28 Conclusions and Design Insights

The experiments presented throughout this chapter investigated FFT butterfly acceleration on the PULP platform using multiple integration strategies and communication architectures. The objective was not only to accelerate the butterfly computation itself, but also to evaluate the impact of system-level communication overhead on the effectiveness of hardware acceleration.

The AXI-based experiments demonstrated that significant performance improvements can be achieved through careful optimization of the software-hardware interaction. Starting from a baseline latency of 59 cycles, successive communication and control optimizations reduced the execution time to 18 cycles while preserving the same butterfly arithmetic implementation. These results confirmed that communication overhead can dominate the overall execution time even when the computational core itself is highly efficient.

A second set of experiments investigated direct integration through the TCDM interconnect. By reducing protocol overhead and shortening the communication path between software and hardware, the optimized TCDM implementation achieved a latency of 11 cycles. This result represents the lowest latency obtained among the investigated integration approaches while maintaining identical butterfly functionality.

The collected results indicate that the computational latency of the butterfly accelerator represents only a small fraction of the total execution time. Although the arithmetic core requires only a few clock cycles to complete the butterfly operation, additional cycles are consumed by data movement, synchronization, protocol handling, and software interaction. Consequently, accelerator performance is influenced by the surrounding infrastructure.

These observations highlight an important HW/SW co-design principle: improving arithmetic performance alone is insufficient to guarantee system-level acceleration. The effectiveness of a hardware accelerator depends equally on how efficiently operands and results can be exchanged with the processing system.

The limitations identified throughout this study motivated the architectural evolution presented later in "**Hardware II: HWPE**" and "**Hardware III: System Pipeline**". Rather than treating the FFT butterfly as an external accelerator accessed through software-managed transactions, the subsequent architecture moves computation closer to the data path and reduces the software involvement required during execution. This approach aims to minimize communication overhead while preserving the computational advantages of dedicated hardware acceleration.

Implementation	Communication Architecture	Latency (Cycles)
Pure Software Implementation	Software Execution	13
AXI Experiment 1	Baseline AXI Integration	59
AXI Experiment 2	Communication-Optimized AXI	24
AXI Experiment 3	Fully Optimized AXI	18
TCDM Experiment 1	Initial TCDM Integration	48
TCDM Experiment 2	Fully Optimized TCDM	11
Radix-8 Butterfly Accelerator	TCDM Integration	TBD

Table 1.17: Summary of FFT butterfly acceleration results.

The software-only reference implementation is described in "**Firmware**". Together, these results provide the basis for the system-level architectural decisions analyzed in "**Hardware II: HWPE**", "**Hardware III: System Pipeline**", and "**System Performance Analysis**".

2 References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-time signal processing*, 3rd ed. Upper Saddle River: Pearson, 2010.
- [2] D. Schiavone, "Understanding and working with PULP," [Online]. Available: https://www.pulp-platform.org/pulp_training.html
- [3] R. Balas and M. Eggimann, "A Deep Dive into HW/SW Development with PULP," [Online]. Available: https://www.pulp-platform.org/pulp_training.html